

路由器项目规划书

1.任务要求

1.基本

1. 转发数据包
2. 自动获取对方 mac
3. 终端控制功能
4. 过滤指定 IP 报文
5. IP 过滤配置文档有效

2.扩展

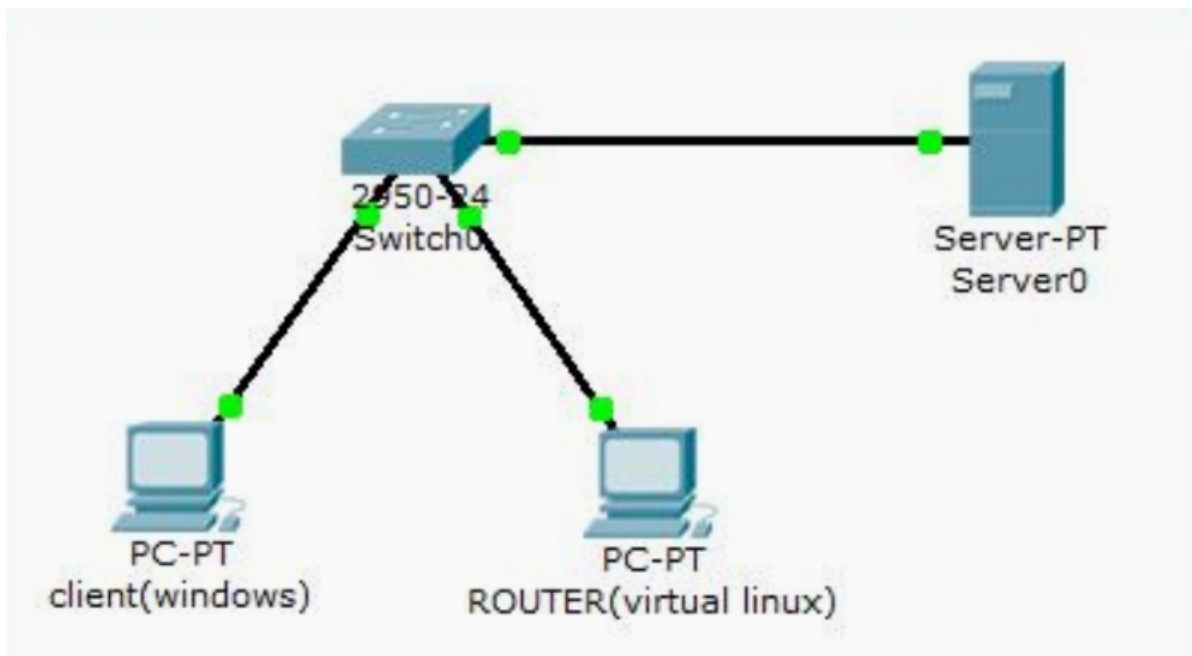
1. 防火墙功能
 - a) 过滤 port
 - b) 过滤 tcp/udp
 - c) 过滤关键字
2. 远程配置 (udp/tcp 通信)
3. 下载/上传配置文档
4. 在线升级功能
5. 提高性能 (线程池)
6. 多级路由器级联

□ 应用知识

1. 原始套接字
2. 数据链路层数据包的抓取 (PF_PACKET、SOCK_RAW)
3. MAC 地址组装/拆解
4. 防火墙的设置
 - a) 设置 Linux 内核协议栈对收到的数据不做处理
 - b) 通过自己编写的程序去收取、处理、转发数据到目的主机
5. 文件的操作
6. 指针、数组、链表应用

2.模拟图

1.模拟图



2. Client

- a) 通过 udp 来完成对 router 的控制
- b) 正常访问 Server 的数据
- c) 10.0.108.11

3. ROUTER

- a) 在 vmware 中开启 2 个网卡，分别为 eth0(当做内网 10.0.108.12) eth1(当做外网 192.168.1.254)
- b) 利用原始套接字对链路层进行数据接收发送
- c) 维护 arp 缓存表
- d) 对接收到的数据进行过滤
- e) 修改目的以及源 MAC

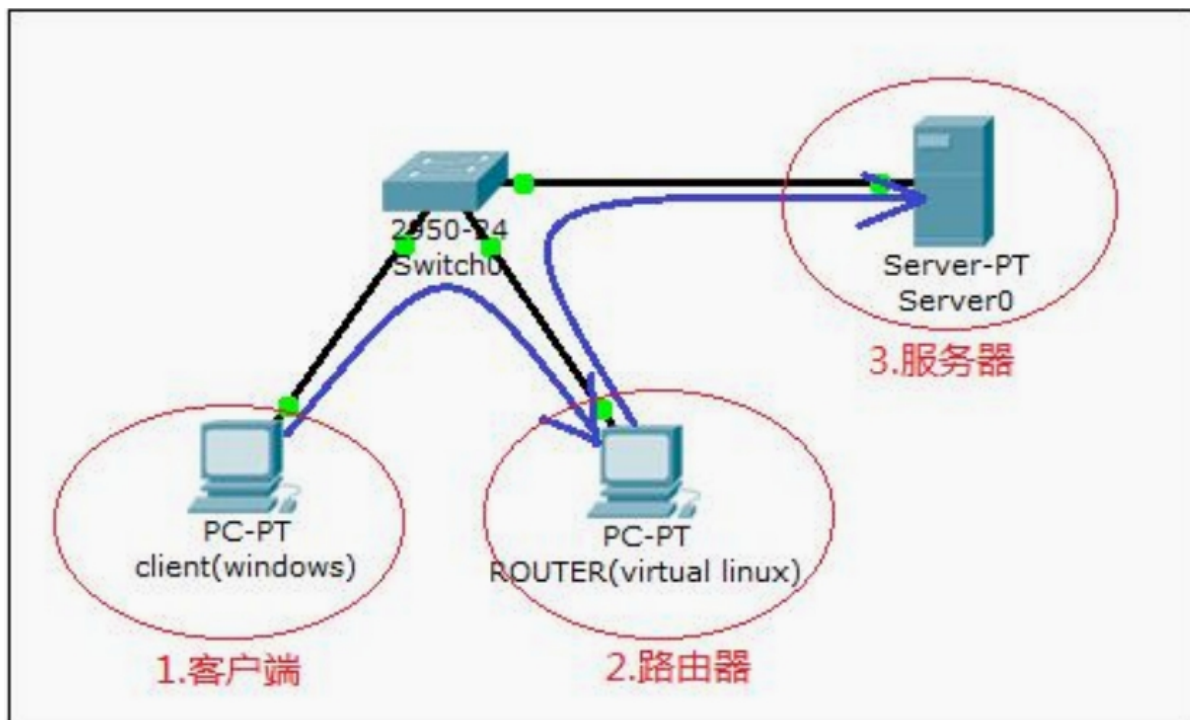
4. Server (选做)

- a) 可以运行一个 WEB 服务器，用于检测 ROUTER 的性能
- b) 192.168.1.1

3.过程描述

1.客户端发送 http 或是ping请求

- a)



b) 客户端（配置默认网关为路由器的 ip）

运行浏览器，访问 192.168.1.1

c) 路由器

通过“链路层”接收来自客户端的数据请求

按配置要求，进行过滤检查

修改 MAC 地址（目的、源）

然后把数据转发到服务器

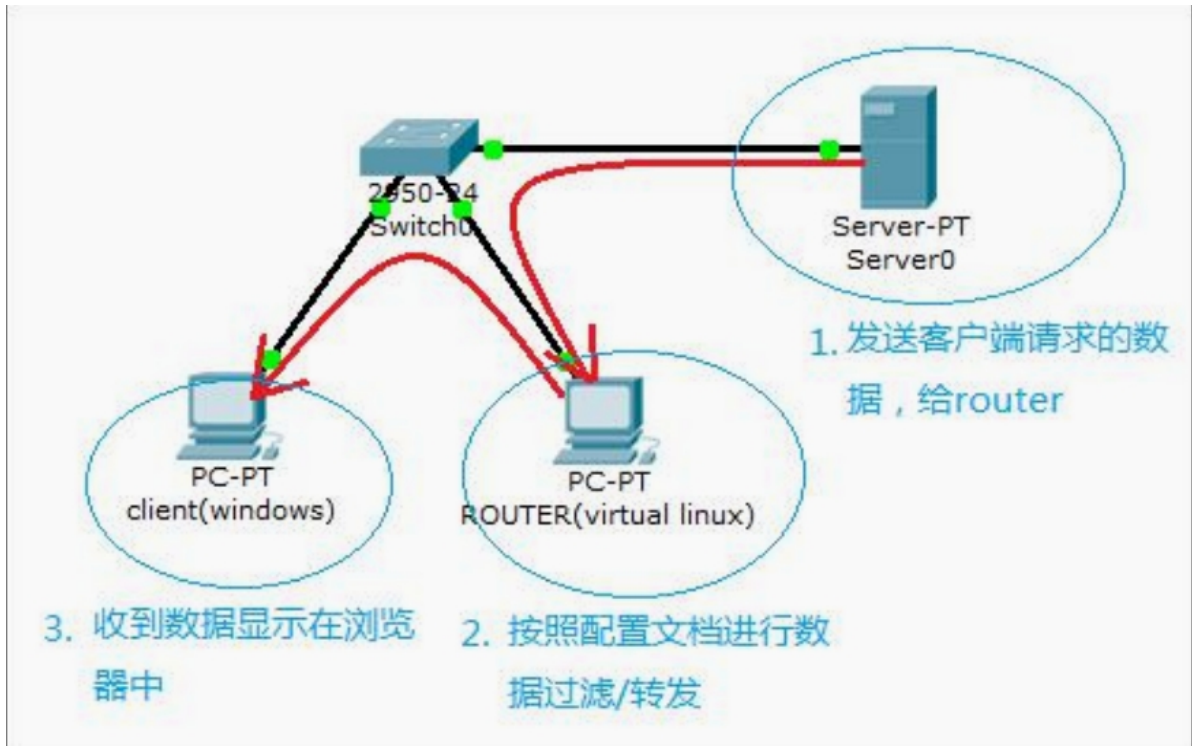
d) 服务器

接收来自路由器的 http或ping 请求

发送 html 文件数据给路由器，或是ttl相应数据

2.服务器返回 http 请求数据

a)



b) 数据传送的过程与 client 发送数据包给 server 基本一样，请参考“客户端发送 http 请求

3.要求效果

1. 在 pc 机上开启虚拟机，充当 router；eth0(10.0.108.12),eth1(192.168.1.254)，实现10.0.108.11 和 192.168.1.1 互相 ping 通。
2. 在 client 端(win11或是Linux)，打开浏览器，输入跨网段的 ip，即 server 的 ip
3. 在 server 端，运行 router 程序，其控制端的效果如下图

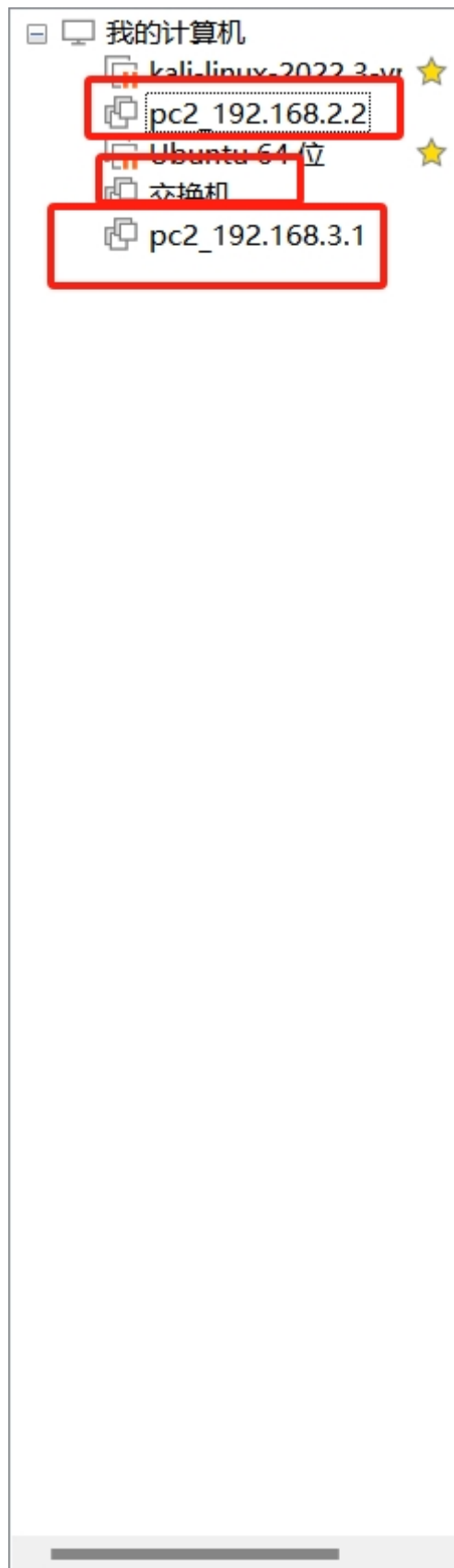


4. 检测 client 端中浏览器的效果
5. 在 server 端，可以输入命令数，来完成相应的功能，查看 arp 缓存表

```
-----key_pthread-----  
=====  
=---setip:172.20.226.4---=  
= 系统设置 : 172.20.226.4 ---=  
= save set -----=  
=---showip-----=  
=---showarp-----=  
=====  
showarp  
  
-----arp_link_start-----  
10.0.108.254-->00:0c:29:41:70:0d  
-----arp_link_end-----
```

2.教程案例：

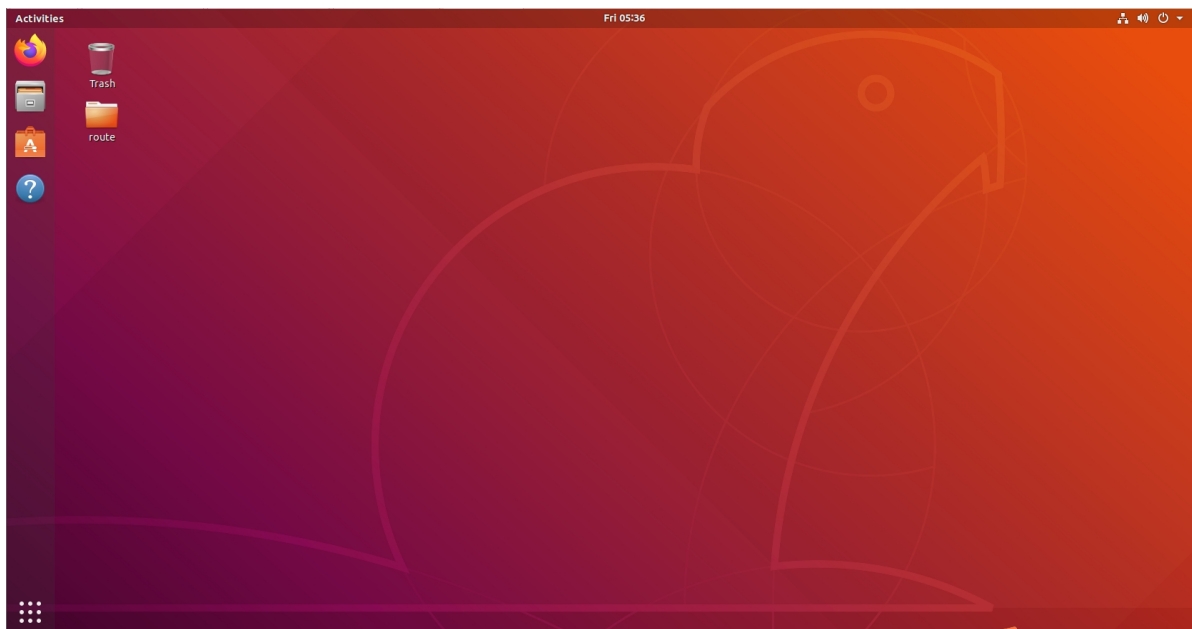
1.使用虚拟机配置两个客户端网口，配置一个路由器



为了区分路由器和客户端，路由器选择ubuntu18版本，客户端选择kali版本

2.路由器配置

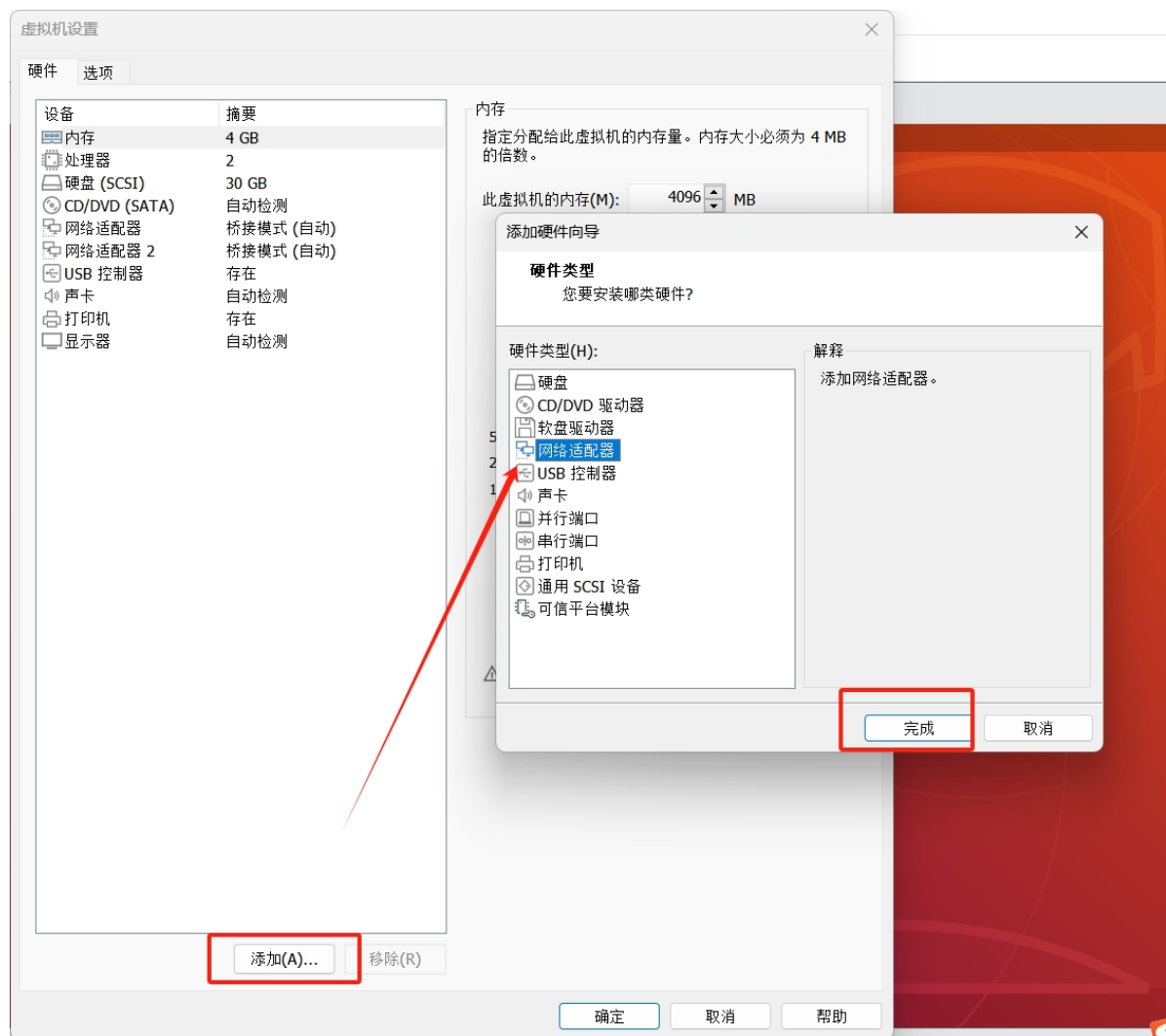
虚拟机使用Ubuntu18版本



设置虚拟机

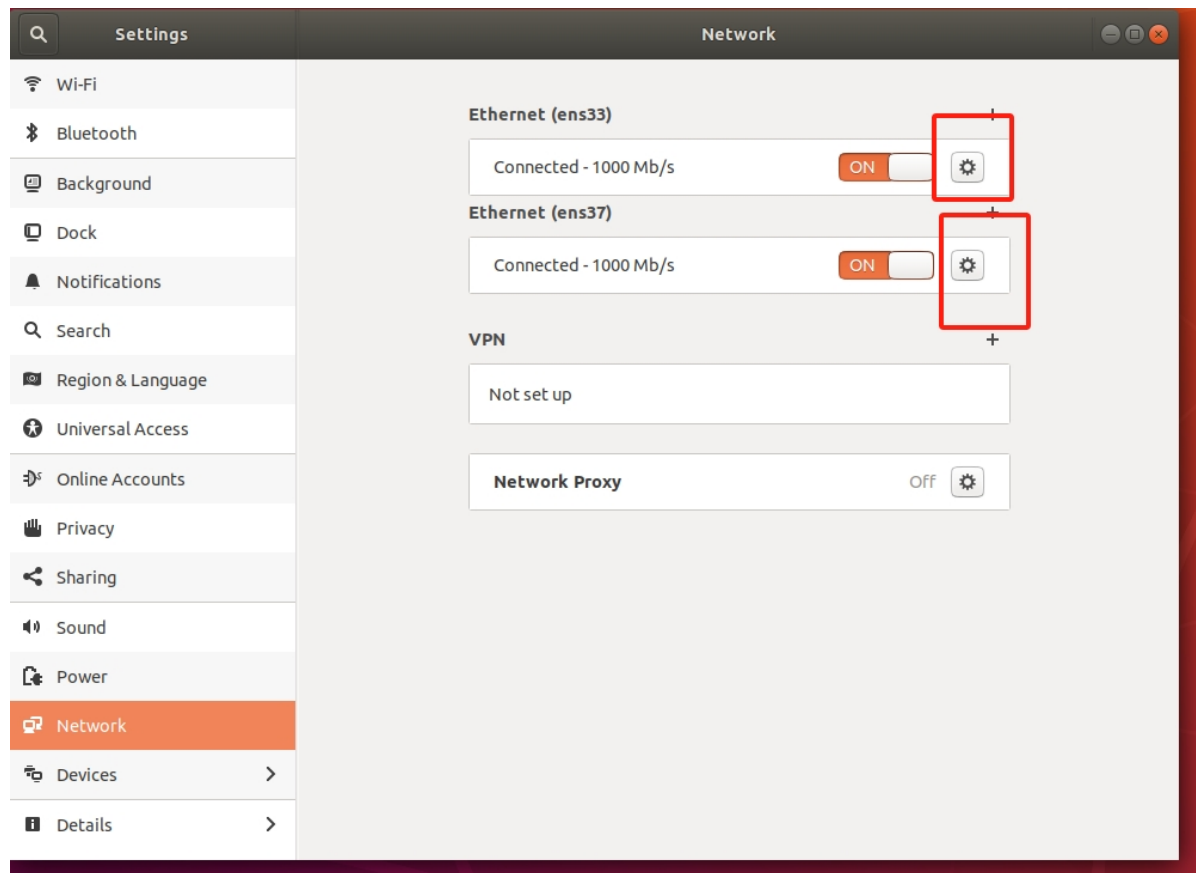


添加网络适配器

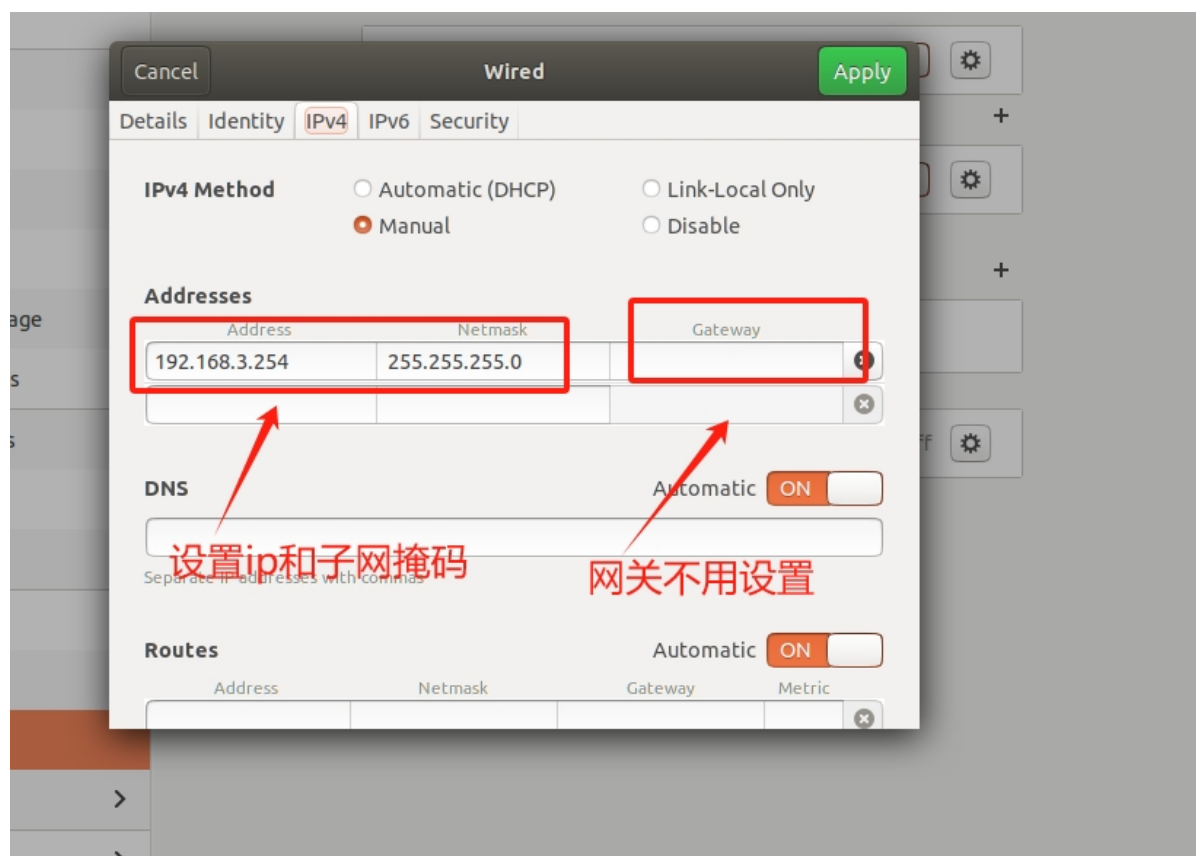


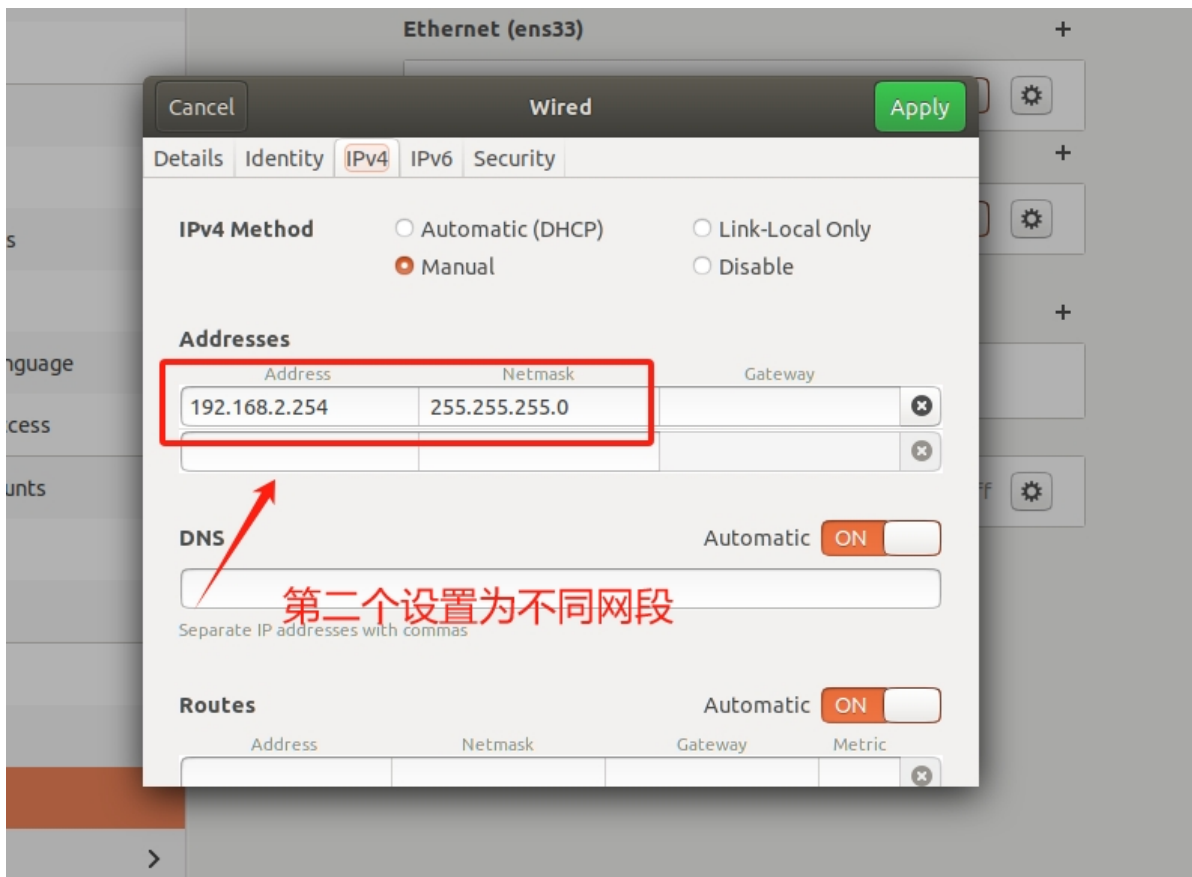
两个网络适配器配置为桥接模式

选择设置

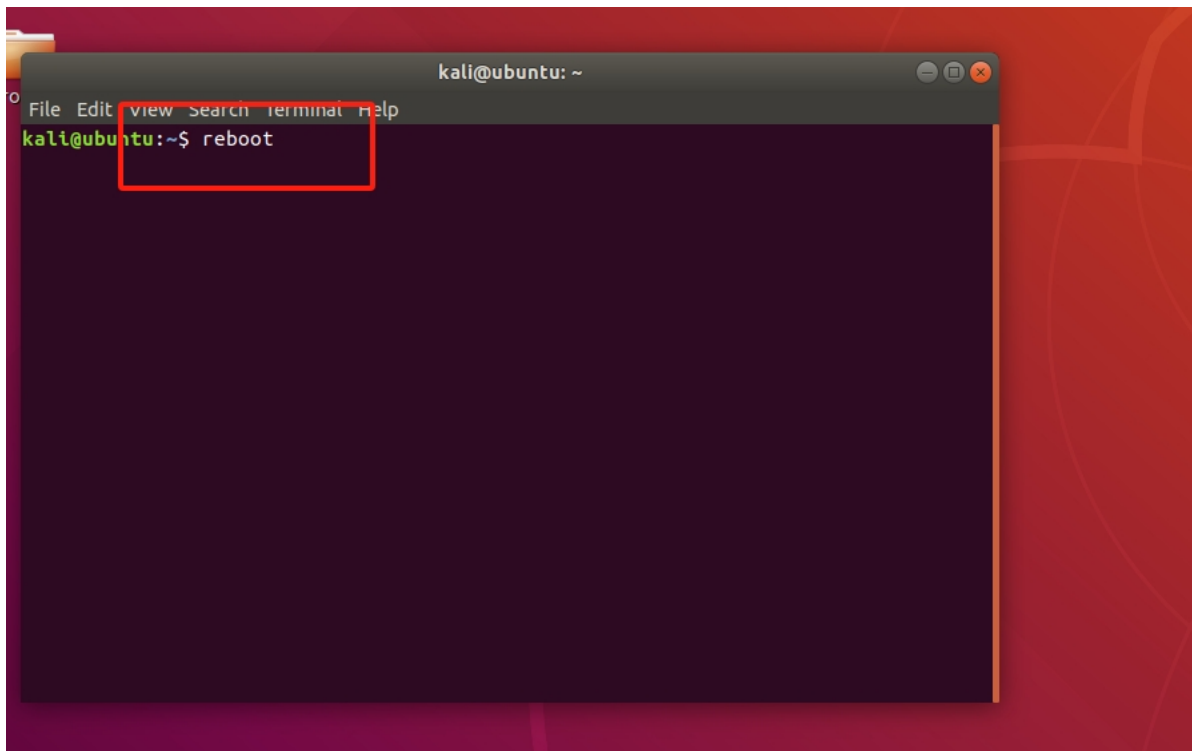


将两个ip地址设置为不同网段





配置完成reboot重启虚拟机



2.客户端配置方法如下

3.客户端配置

1.保证虚拟机为关机状态

orkstation

选项卡(O) 帮助(H)

pc2_192.168.2.2

pc2_192.168.2.2

▶ 开启此虚拟机

🔒 锁定虚拟机设置

🔧 升级此虚拟机

▼ 设备

内存

2 GB

处理器

4

硬盘 (SCSI)

80.1 GB

CD/DVD (IDE)

自动检测

网络适配器

桥接模式 (自动)

USB 控制器

存在

声卡

自动检测

显示器

自动检测

▼ 描述

Kali Rolling (2022.3) x64

2022-08-08

Username: kali

Password: kali

(US keyboard layout)

* Kali Homepage:

<https://www.kali.org/>

* Kali Documentation:

<https://www.kali.org/docs/>

* Kali Tools:

<https://www.kali.org/tools/>

* Forum/Community Support:

▼ 虚拟机详细信息

状态: 已关机

快照: pc2_192.168.3.1 的克隆的快照

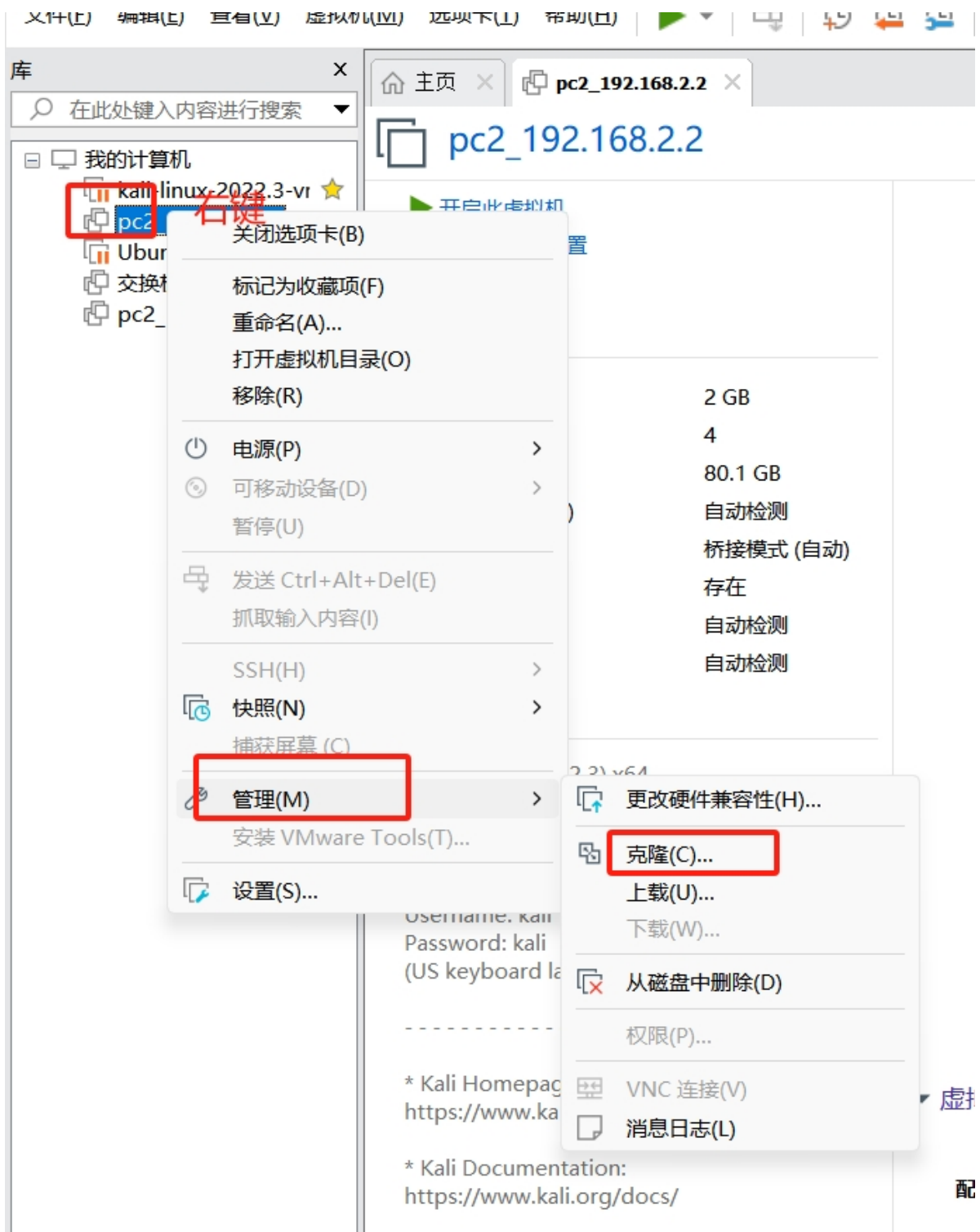
配置文件: D:\vm\clon\2\kali-linux-2022.3-vmware-amd64 的克隆.vmx

克隆自: D:\vm\kali-linux-2022.3-vmware-amd64\vmwarevm\kali-linux-2022.3-vmware-amd64.vmx

硬件兼容性: Workstation 10.x 虚拟机

主 IP 地址: 网络信息不可用

3.克隆虚拟机



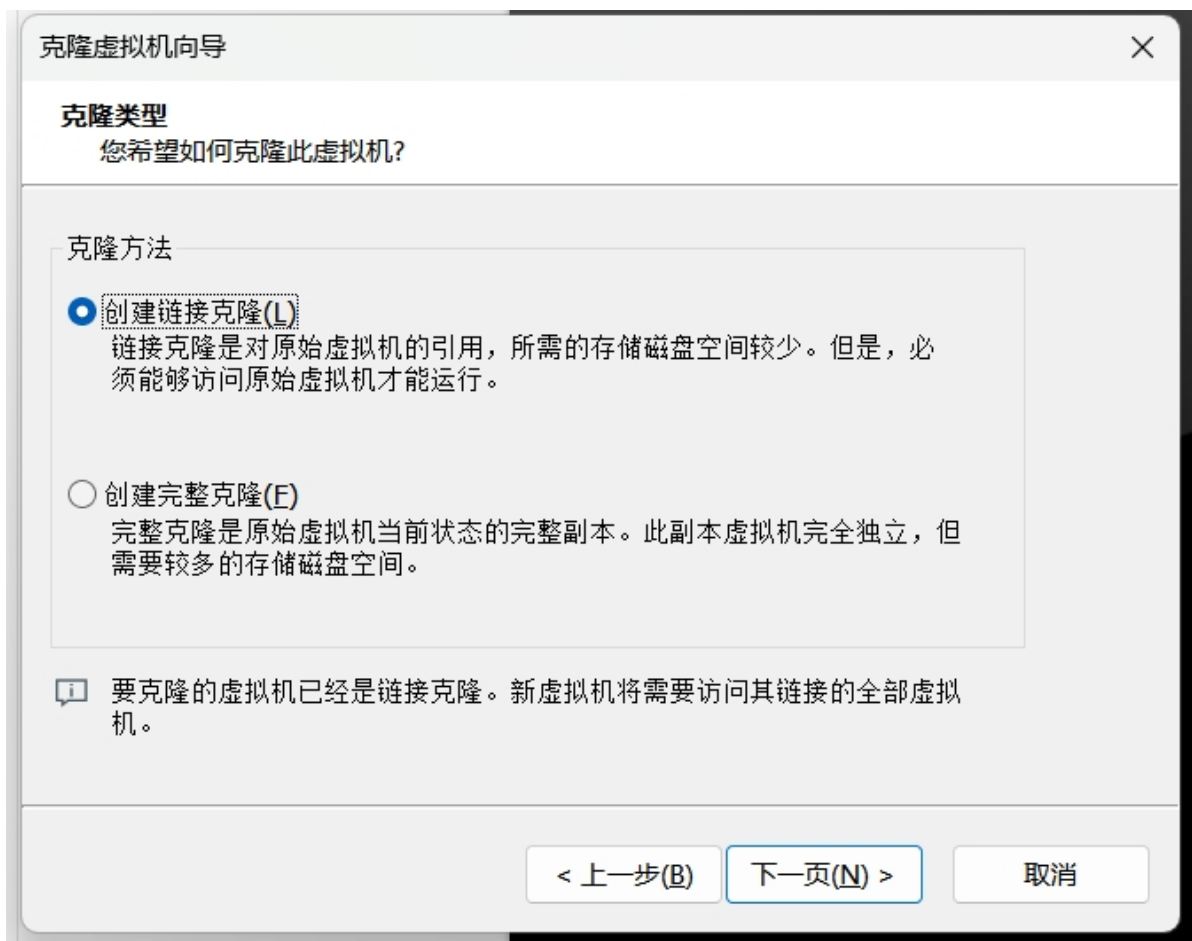
选择克隆



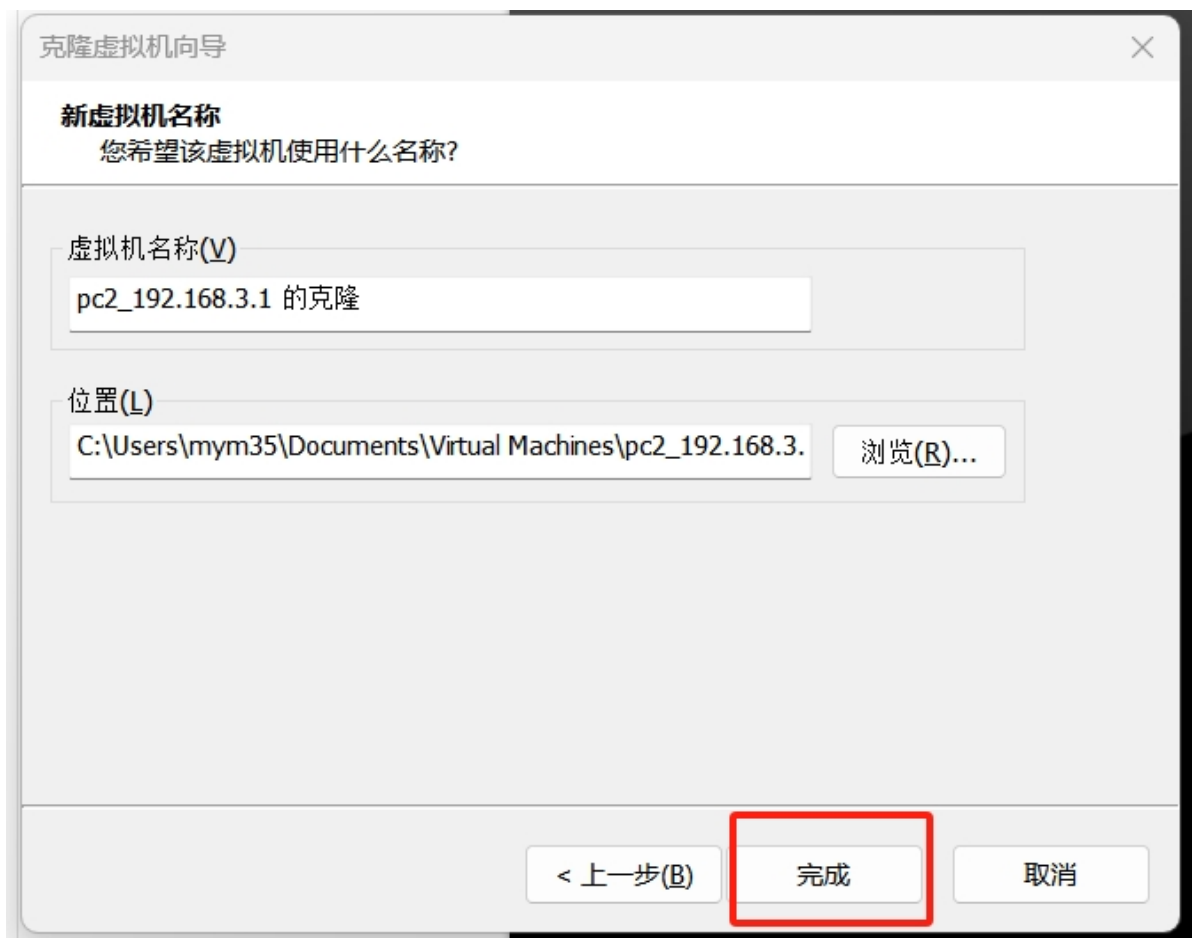
选择下一页



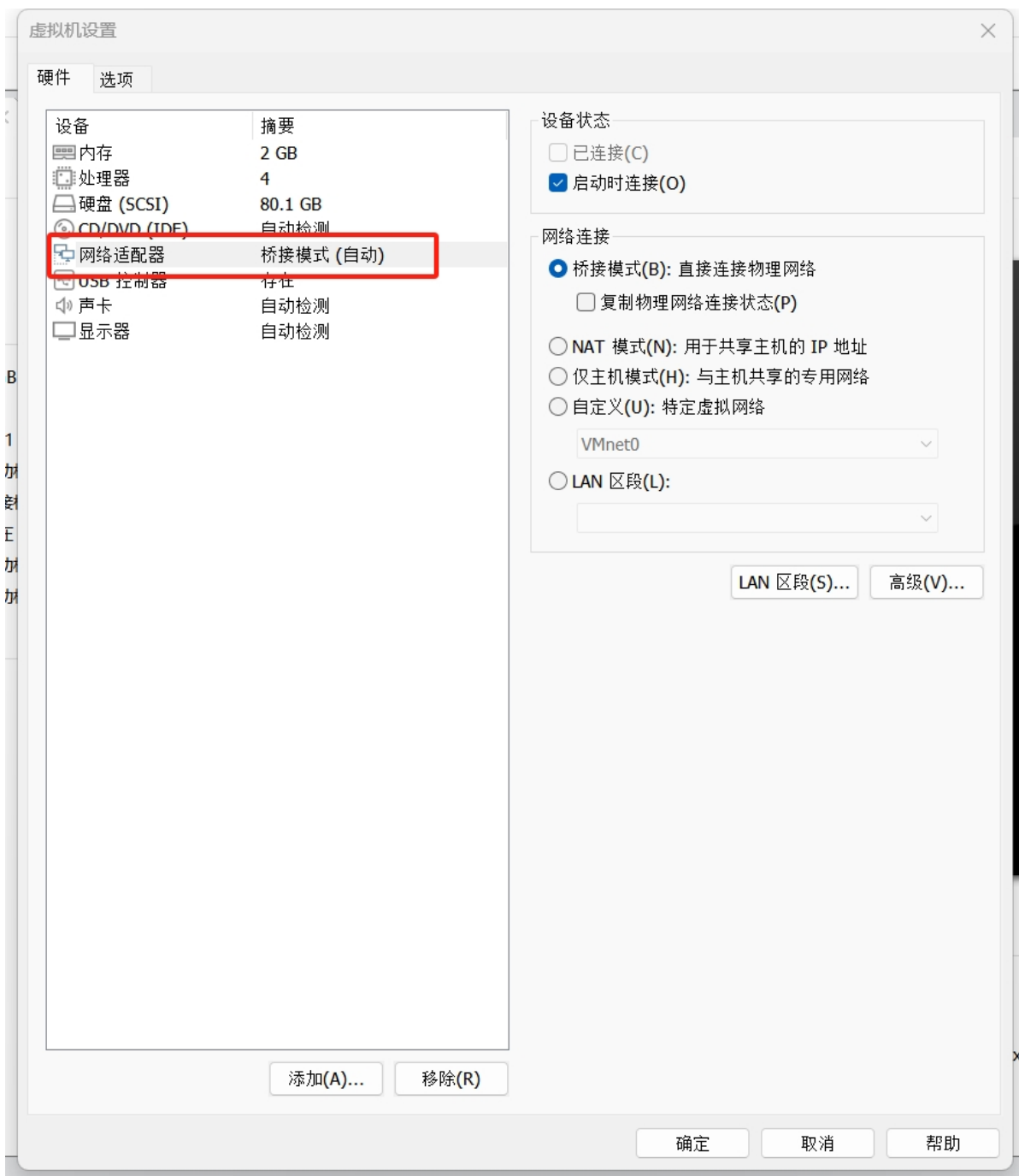
创建克隆连接



完成即可



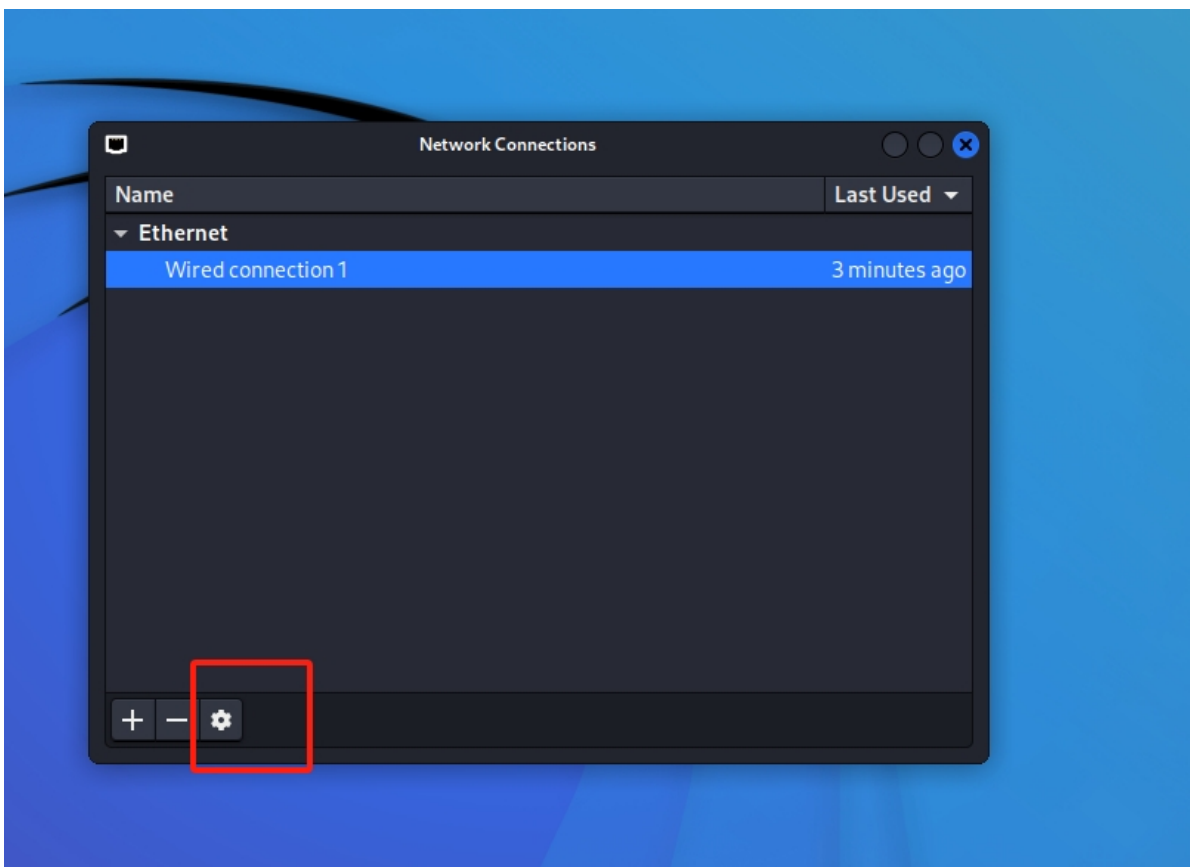
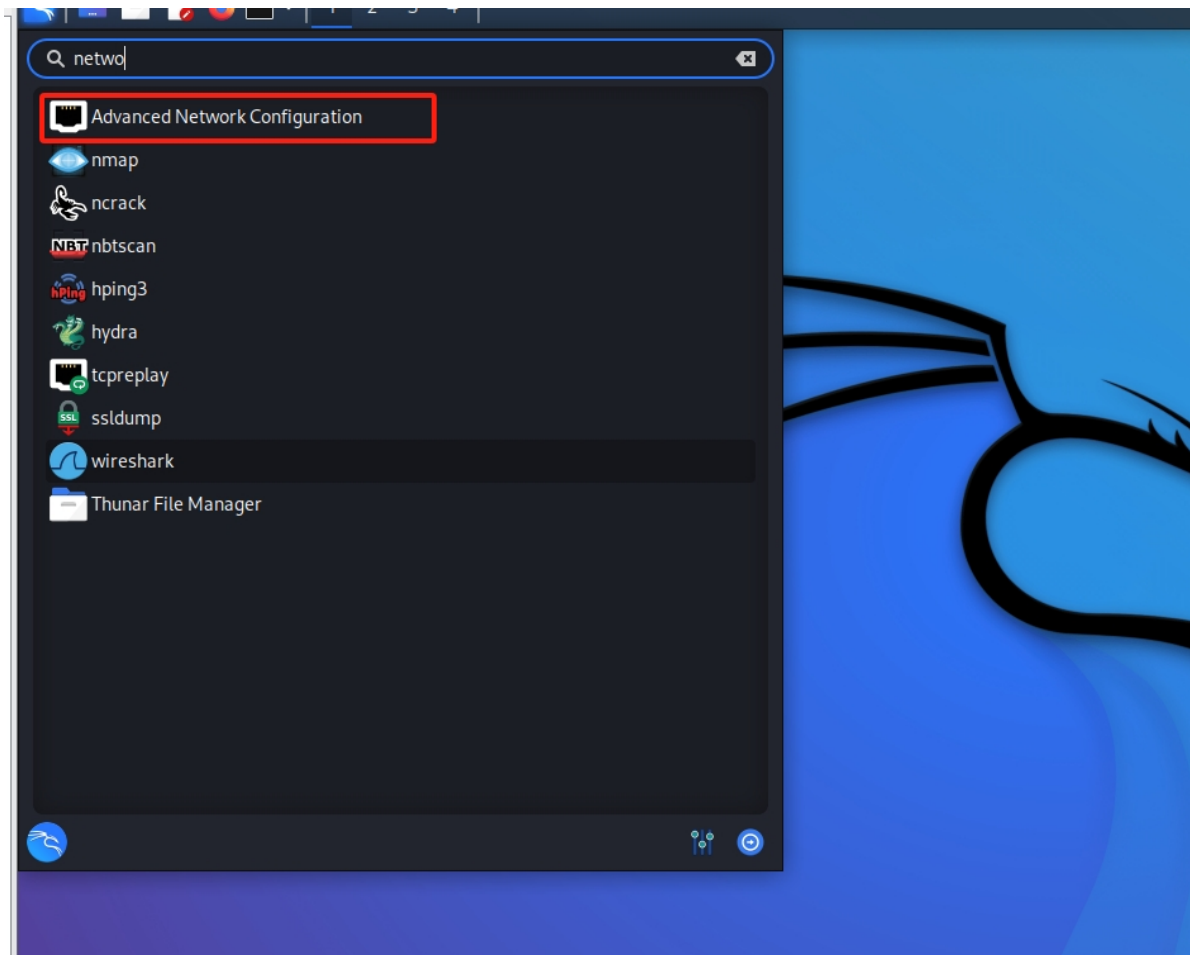
在客户端设置中将网络模式配置为桥接模式



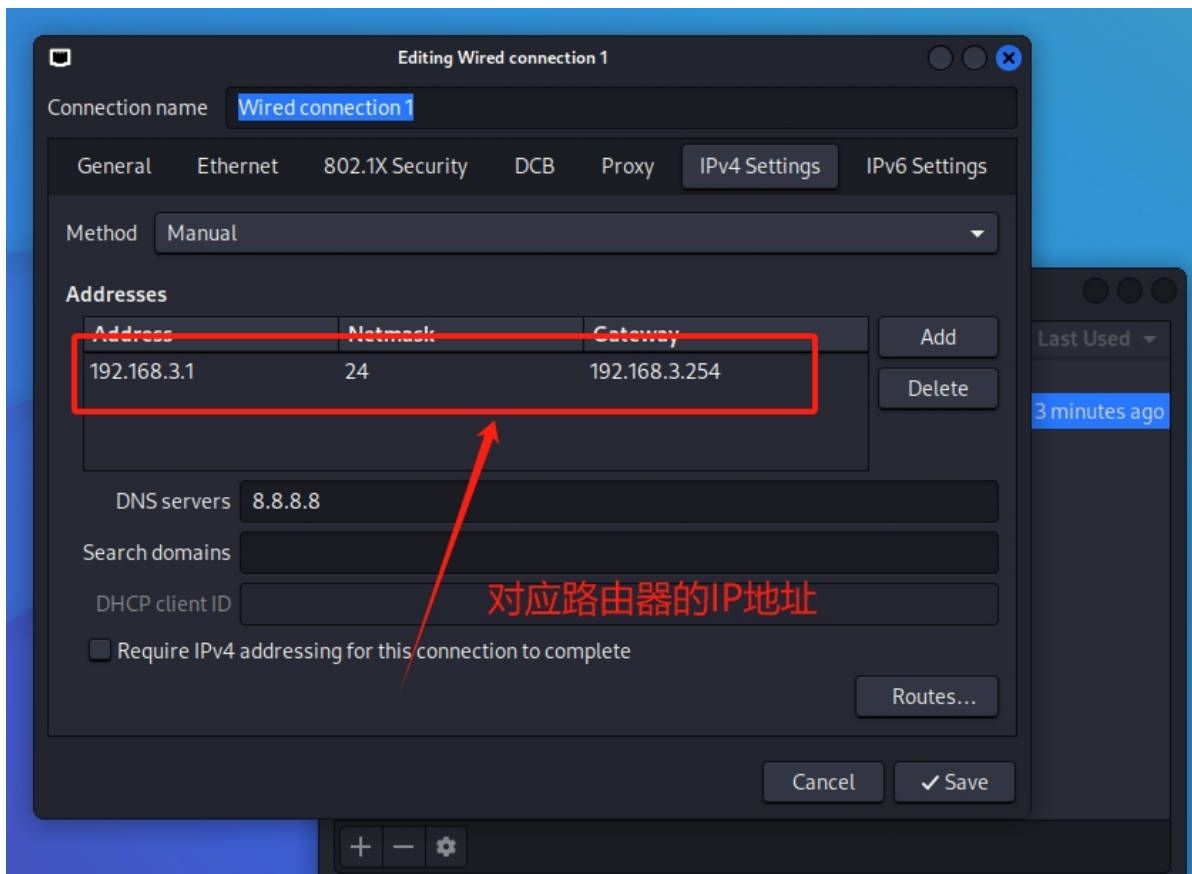
启动虚拟机



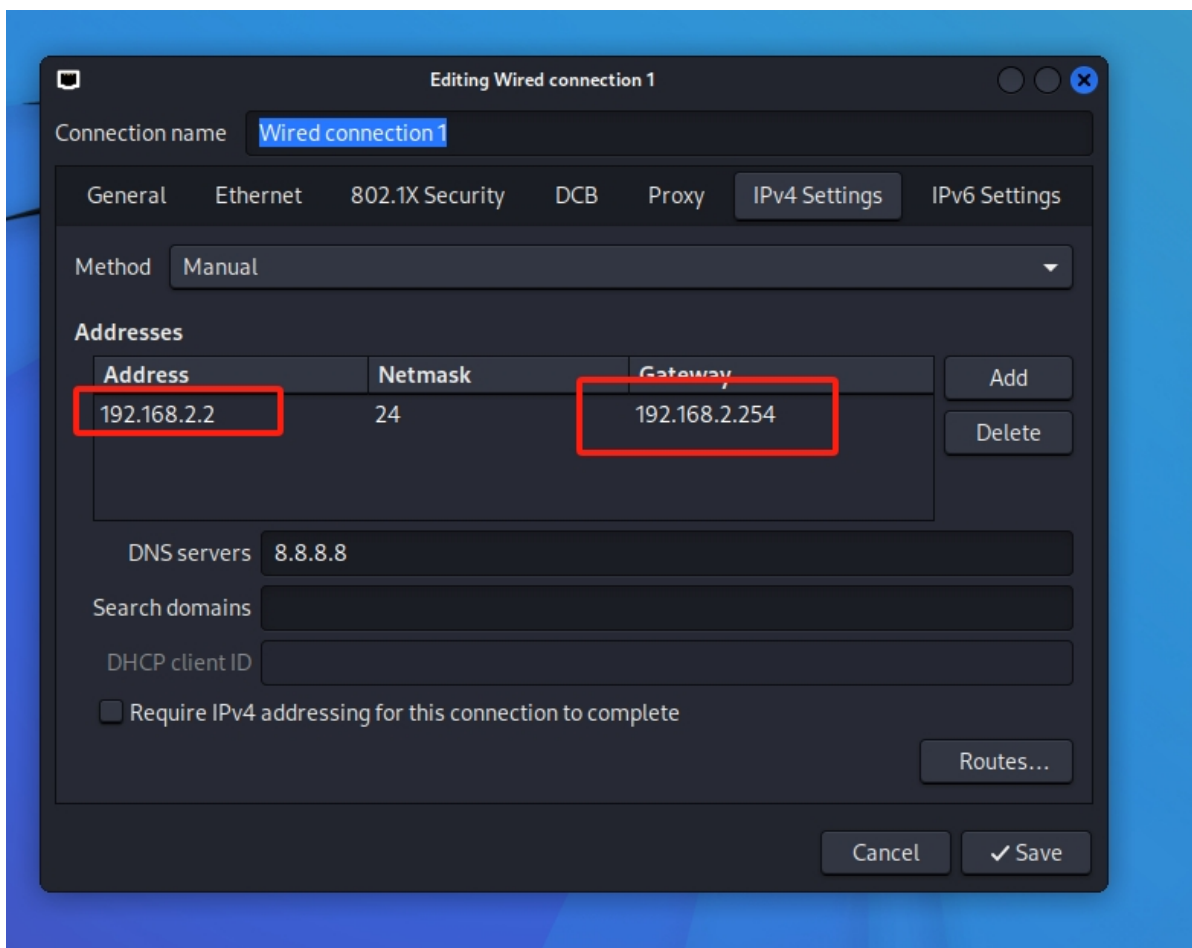
配置客户端ip, 两个客户端ip地址的网关, 分别要对应路由器的两个ip地址



第一个客户端



第二个客户端



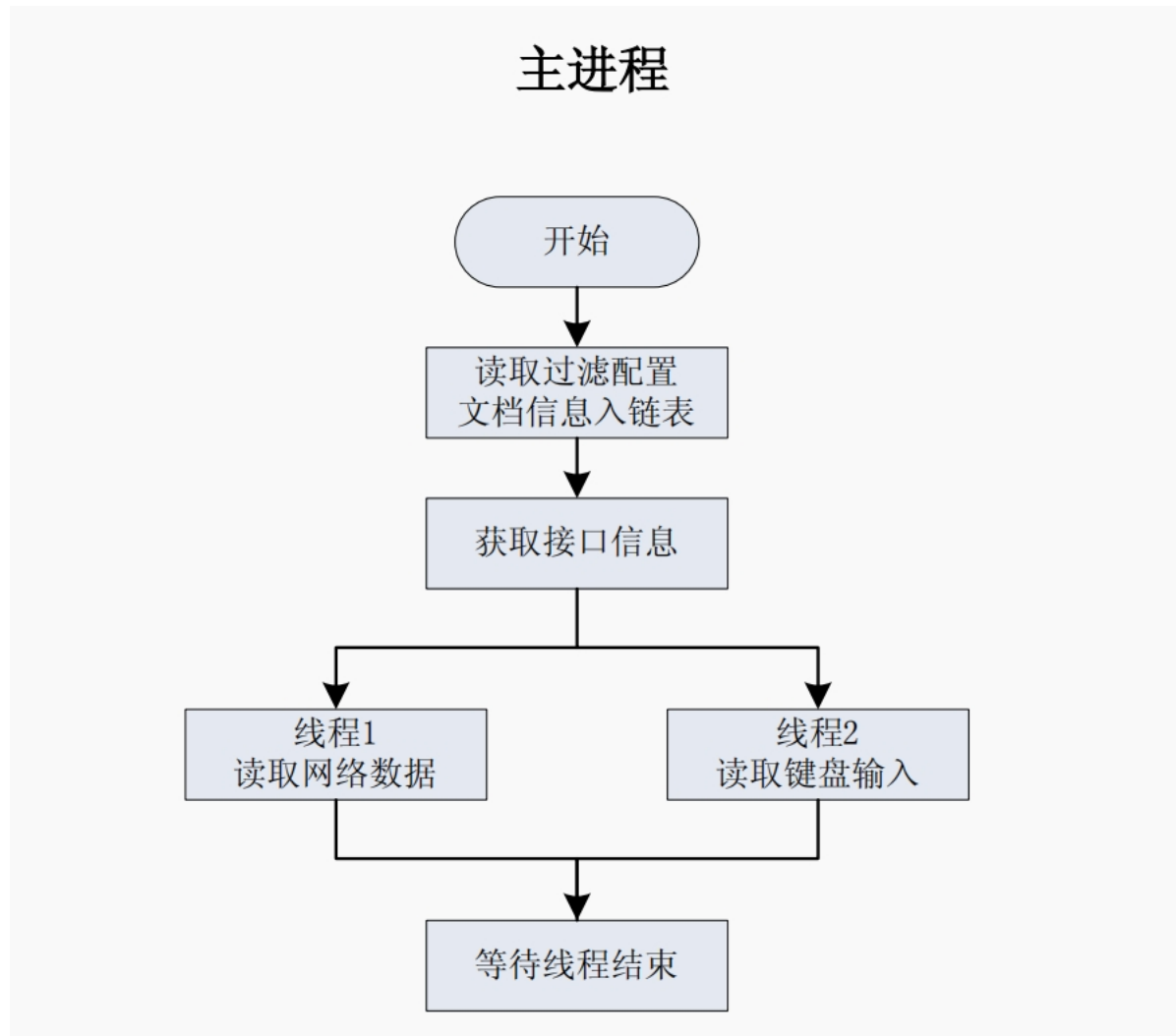
注意两个客户端的ip地址和网关都要配置，分别对应路由器配置的两个ip地址

配置工作完成，下面进入编程环节

4.编程

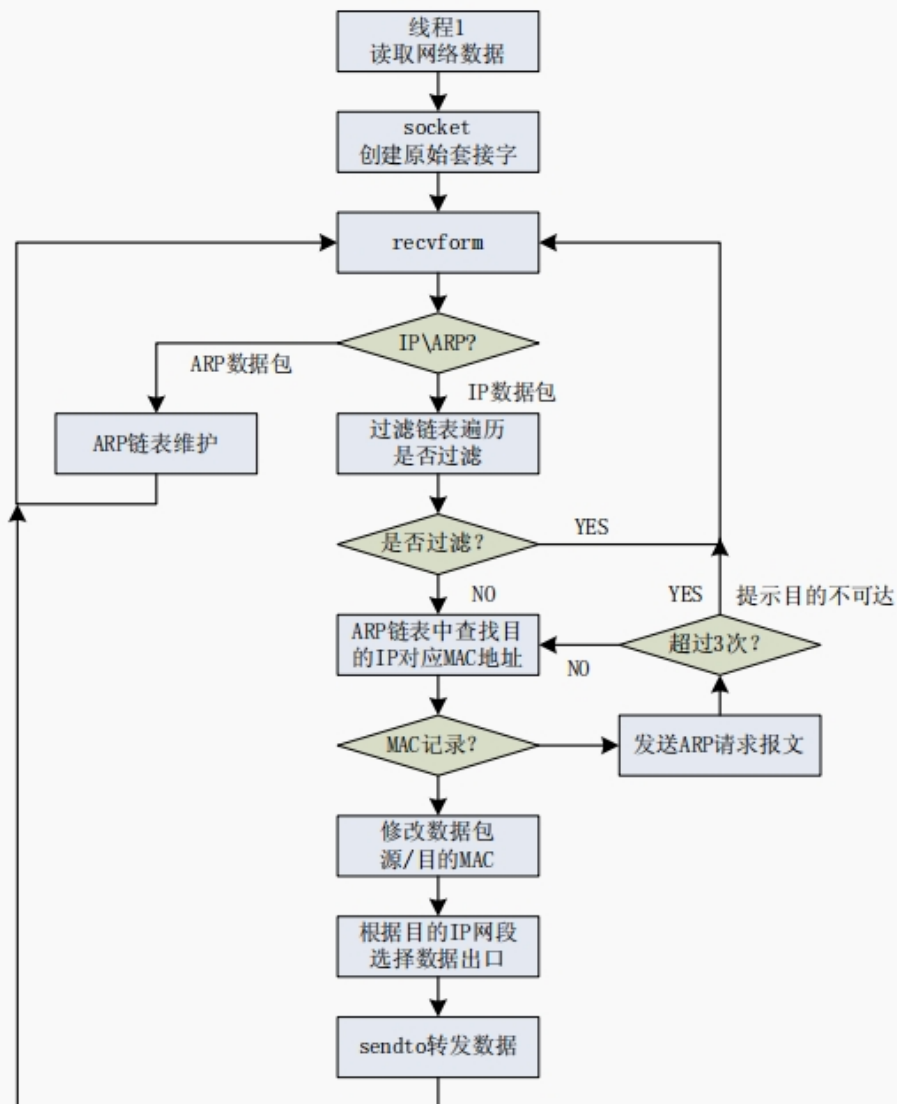
1.系统流程图

主进程读取文档信息，配置过滤链表，获取端口信息，启动两个线程



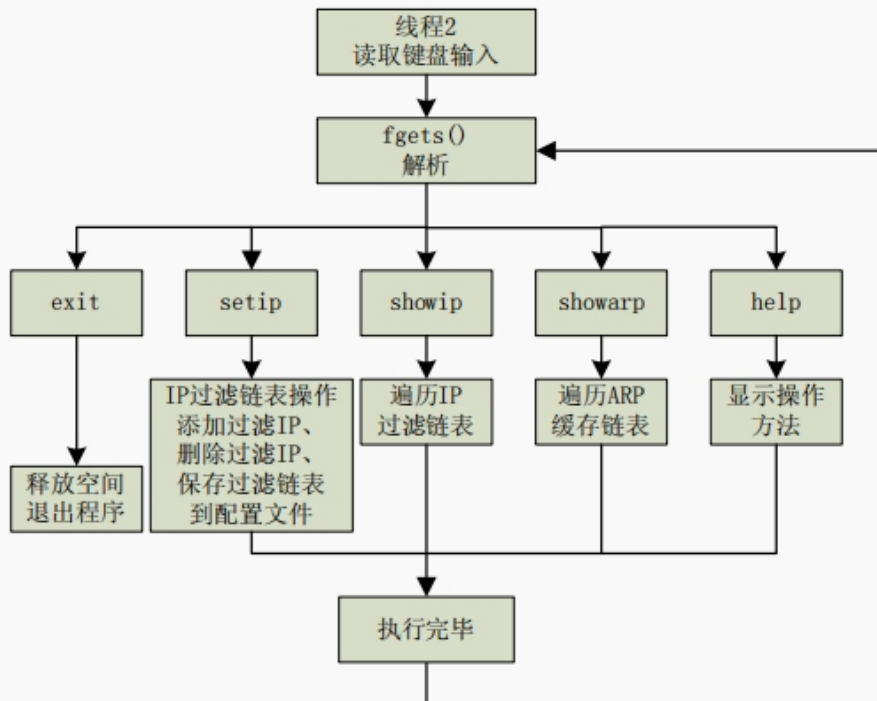
线程1读取网络数据

线程1 读取网络数据



线程2，读取键盘输入信息

线程 2 读取键盘输入



程序内容如下

main.c内容，主要配置文件，获取接口信息，启动线程key_thread、线程arp_thread、线程ip_thread，进行socket连接，循环接收缓冲区的内容

```
//初始化 配置文件
init_ip_link();
//获取接口信息
getinterface();
//创建键盘处理函数并脱离
pthread_t KEY_T;
pthread_create(&KEY_T, NULL, key_thread, NULL);
pthread_detach(KEY_T);

//创建原始套接字,接收所有类型的数据包
raw_sock_fd = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
if(raw_sock_fd<=0){
    perror("socket");
    _exit(-1);
}
char recv_buff[RECV_SIZE]="";//原始套接字数据包大约为1500个字节
```

```

ssize_t recv_len=0;
while(1){
    bzero(recv_buff,sizeof(recv_buff));
    recv_len = recvfrom(raw_sock_fd, recv_buff, sizeof(recv_buff), 0, NULL,
NULL);
    if(recv_len<=0||recv_len>RECV_SIZE){
        perror("recvfrom");
        continue;
    }
    if((recv_buff[12]==0x08)&&(recv_buff[13]==0x06)){//ARP协议包
        ARP_LINK *p = (ARP_LINK *)malloc(sizeof(ARP_LINK));
        if(p==NULL){
            perror("malloc");
            continue;
        }
        memcpy(p->mac, recv_buff+22, 6);//mac
        memcpy(p->ip , recv_buff+28, 4);//ip
        //printf("%d.%d.%d.%d-->",p->ip[0],p->ip[1],p->ip[2],p->ip[3]);
        pthread_t ARP_T;
        pthread_create(&ARP_T, NULL,arp_thread, (void*)p);
        pthread_detach(ARP_T);
    }
    if((recv_buff[12]==0x08)&&(recv_buff[13]==0x00)){//IP协议包
        //目的ip过滤
        IP_LINK *ip_pb = find_ip(ip_head, (unsigned char*)recv_buff+30);
        if(ip_pb!=NULL){
            continue;
        }
        RECV_DATA *recv = (RECV_DATA *)malloc(sizeof(RECV_DATA));
        recv->data_len = recv_len;
        memcpy(recv->data, recv_buff, recv_len);
        //创建转发数据包处理线程
        pthread_t IP_T;
        pthread_create(&IP_T, NULL,ip_thread, (void*)recv);
        pthread_detach(IP_T);
    }
}
}

```

详细内容见main.c

key_thread.c线程, 主要实现以下, key_cmd的命令

```

KEY_CMD key_cmd[]={
    {"help",help},
    {"setip",setip},
    {"delip",delip},
    {"showip",showip},
    {"showarp",showarp},
    {"saveset",saveset},
    {"exit",exit_route}
};

/*****
函 数: void *key_thread(void *arg)
功 能: 键盘输入处理线程
参 数: 无
返回值: 无
*****/

```

```

void *key_pthread(void *arg){
    printf("-----key_pthread-----\n");
    help("--");
    while(1){
        char buff[100]="";
        char cmd[100]="";
        char msg[100]="";
        fgets(buff,sizeof(buff),stdin);
        buff[strlen(buff)-1]='\0';
        sscanf(buff,"%[^:]:%s",cmd,msg);
        int i;
        for(i=0;i<(sizeof(key_cmd)/sizeof(KEY_CMD));i++){
            if(strcmp(cmd,key_cmd[i].cmd_str)==0){
                key_cmd[i].fun(msg);
                break;
            }
        }
    }
    return NULL;
}

```

线程2创建arp_thread,添加arp信息

```

void *arp_pthread(void *arg){
    ARP_LINK *p= (ARP_LINK *)arg;
    arp_head = inner_arp_link(arp_head, p);//添加到链表中
    //ARP回应--
    return NULL;
}

```

线程3创建ip_thread,用来完成ip相关数据的整理和转发

```

int netmask_num =3;//默认3个255 掩码的长度

//ARP结构体
unsigned char arp_buf[42] = {
    0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, // dst MAC [0]-->[5]
    0, 0, 0, 0, 0, 0, // src MAC [6]-->[11]
    0x08, 0x06, // ARP [12]-->[13]
    0x00, 0x01, // Ethernet [14]-->[15]
    0x08, 0x00, // IP [16]-->[17]
    0x06, // HW address size [18]
    0x04, // protocol(IP here) address size
[19]
    0x00, 0x01, // opcode : 1.arp request 2.arp
answer 3.rarp request 4.rarp answer [20]-->[21]
    0, 0, 0, 0, 0, 0, // src MAC [22]-->[27]
    0, 0, 0, 0, // src ip : 192.168.1.254 [28]-->
[31]
    0, 0, 0, 0, 0, 0, // dst MAC [32]-->[37]
    0, 0, 0, 0 // dst ip : [38]-->[41]
};

/*****

```

函 数: int find_network_segment(unsigned char *ip)

功 能: 根据目的ip查找对应网段

参 数: unsigned char *ip 目的IP

返回值: 网段编号

```
*****/
int find_network_segment(unsigned char *ip){
    int i=0,j=get_interface_num();
    for(i=0;i<j;i++){
        if(memcmp(net_interface[i].ip, ip, netmask_num)==0)//比较掩码覆盖的前面几位
            break;
    }
    return i;
}
```

函 数: int send_msg(int network_num,unsigned char *msg_data,unsigned long msg_data_len)

功 能: 根据网卡编号将对应的数据转发出去

参 数: int network_num 网卡标号

unsigned char *msg_data 转发的数据

unsigned long msg_data_len 数据的长度

返回值: 网段编号

```
*****/
int send_msg(char *eth_name,unsigned char *msg_data,unsigned long msg_data_len){
    struct ifreq ethrep;
    strncpy(ethrep.ifr_name, eth_name,IFNAMSIZ);
    if(-1 == ioctl(raw_sock_fd,SIOGIFINDEX,&ethrep)){
        perror("send_msg-ioctl");
        close(raw_sock_fd);
        _exit(-1);
    }
    struct sockaddr_ll sll;
    bzero(&sll,sizeof(sll));
    sll.sll_ifindex = ethrep.ifr_ifindex;
    int len = sendto(raw_sock_fd, msg_data, msg_data_len, 0,(struct
sockaddr*)&sll, sizeof(sll));
    return len;
}
/*****
```

函 数: void creat_arp_msg(unsigned char *ip,int network_num)

功 能: 组件ARP报文

参 数: unsigned char *ip 目的IP

int network_num 网段编号

返回值: 无

```
*****/
void creat_arp_msg(unsigned char *ip,int network_num){
    memcpy(arp_buf+38, ip, 4); //目的IP
    memcpy(arp_buf+28, net_interface[network_num].ip, 4); //源ip
    memcpy(arp_buf+6, net_interface[network_num].mac, 6); //源mac
    memcpy(arp_buf+22, net_interface[network_num].mac, 6); //源mac
}
```

函 数: void *ip_pthread(void *arg)

功 能: IP处理线程

根据对应的网段选择发送的网卡

没有目的IP的MAC地址则发送ARP请求

参 数: void *arg 待转发的数据

返回值：无

```
*****/
void *ip_pthread(void *arg){
    RECV_DATA *msg_data = (RECV_DATA *)arg;
    int network_num = find_network_segment((msg_data->data)+30); //获取目的ip对应的
网卡出口
    if(network_num==get_interface_num()){ //当前路由器还不支持这个网段，涉及到路由表
        //printf("ip_pthread:network_num=%d no this
network_segment\n",network_num);
        //network_num=0; //默认发送给0网卡
        return NULL;
    }
    if(msg_data->data[33]==255){ //广播数据一律不转发
        return NULL;
    }
    if(strcmp(net_interface[network_num].name,"lo")==0){ //本地回环不转发，会死机
        return NULL;
    }
    ARP_LINK * pb = find_arp_from_ip(arp_head,(msg_data->data)+30);
    if(pb==NULL){ //发送arp包
        int j;
        for(j=0;j<3;j++){ //发j次arp请求包，未找到则放弃转发
            creat_arp_msg((msg_data->data)+30,network_num);
            int ret =
send_msg(net_interface[network_num].name,arp_buf,sizeof(arp_buf)); //发送arp消息
            if(ret<0){
                perror("sendmsg1");
            }
            pb = find_arp_from_ip(arp_head,(msg_data->data)+30);
            if(pb != NULL){
                break;
            }
        }
        if(pb==NULL){
            printf("%d.%d.%d.%d arp error\n",msg_data->data[30],msg_data-
>data[31],msg_data->data[32],msg_data->data[33]);
            goto IP_PTHREAD_END;
        }else{
            printf("%d.%d.%d.%d arp OK\n",msg_data->data[30],msg_data-
>data[31],msg_data->data[32],msg_data->data[33]);
        }
    }
    //转发
    memcpy((msg_data->data), pb->mac, 6); //修改源MAC
    memcpy((msg_data->data)+6, net_interface[network_num].mac, 6); //修改目的MAC
    int ret = send_msg(net_interface[network_num].name,msg_data->data,msg_data-
>data_len); //发送消息
    if(ret<0){
        printf("msg_data.data_len=%d\n",msg_data->data_len);
        perror("sendmsg2");
    }

IP_PTHREAD_END:
    return NULL;
}
```

创建ip_link,保存链表信息到配置文件

```
void save_ip_link(){
    FILE *ip_config = fopen(ip_config_name,"wb+");
    if(ip_config == NULL){
        perror("!!!configure file,in main.c");
        _exit(1);
    }
    char buff[20]="";
    IP_LINK *pb=ip_head;
    while(pb!=NULL){
        sprintf(buff,"%d.%d.%d.%d\n",pb->ip[0],pb->ip[1],pb->ip[2],pb->ip[3]);
        fputs(buff,ip_config);
        pb = pb->next;
    }
    fclose(ip_config);
}
```

创建get_interface.c的文件, 获取网络接口信息

```
int interface_num=0;//接口数量
INTERFACE net_interface[MAXINTERFACES];//接口数据

/*****
函 数: int get_interface_num()
功 能: 获取接口数量
参 数: 无
*****/
int get_interface_num(){
    return interface_num;
}

/*****
函 数: int getinterface()
功 能: 获取接口信息
参 数: 无
*****/
void getinterface(){
    struct ifreq buf[MAXINTERFACES]; /* ifreq结构数组 */
    struct ifconf ifc; /* ifconf结构 */

    int sock_raw_fd = socket(PF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    /* 初始化ifconf结构 */
    ifc.ifc_len = sizeof(buf);
    ifc.ifc_buf = (caddr_t) buf;

    /* 获得接口列表 */
    if (ioctl(sock_raw_fd, SIOCGIFCONF, (char *) &ifc) == -1){
        perror("SIOCGIFCONF ioctl");
        return ;
    }
    interface_num = ifc.ifc_len / sizeof(struct ifreq); /* 接口数量 */
    printf("interface_num=%d\n\n", interface_num);
    char buff[20]="";
    int ip;
    int if_len = interface_num;
    while (if_len-- > 0){ /* 遍历每个接口 */
```

```

printf("%s\n", buf[if_len].ifr_name); /* 接口名称 */
sprintf(net_interface[if_len].name, "%s", buf[if_len].ifr_name); /* 接口
名称 */

printf("-%d-%s--\n", if_len, net_interface[if_len].name);
/* 获得接口标志 */
if (!(ioctl(sock_raw_fd, SIOCGIFFLAGS, (char *) &buf[if_len]))){
    /* 接口状态 */
    if (buf[if_len].ifr_flags & IFF_UP){
        printf("UP\n");
        net_interface[if_len].flag = 1;
    }
    else{
        printf("DOWN\n");
        net_interface[if_len].flag = 0;
    }
}
else{
    char str[256];
    sprintf(str, "SIOCGIFFLAGS ioctl %s", buf[if_len].ifr_name);
    perror(str);
}

/* IP地址 */
if (!(ioctl(sock_raw_fd, SIOCGIFADDR, (char *) &buf[if_len]))){
    printf("IP:%s\n", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));
    bzero(buff, sizeof(buff));
    sprintf(buff, "%s", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));
    inet_pton(AF_INET, buff, &ip);
    memcpy(net_interface[if_len].ip, &ip, 4);
}
else{
    char str[256];
    sprintf(str, "SIOCGIFADDR ioctl %s", buf[if_len].ifr_name);
    perror(str);
}

/* 子网掩码 */
if (!(ioctl(sock_raw_fd, SIOCGIFNETMASK, (char *) &buf[if_len]))){
    printf("netmask:%s\n", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));
    bzero(buff, sizeof(buff));
    sprintf(buff, "%s", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));
    inet_pton(AF_INET, buff, &ip);
    memcpy(net_interface[if_len].netmask, &ip, 4);
}
else{
    char str[256];
    sprintf(str, "SIOCGIFADDR ioctl %s", buf[if_len].ifr_name);
    perror(str);
}

/* 广播地址 */
if (!(ioctl(sock_raw_fd, SIOCGIFBRDADDR, (char *) &buf[if_len]))){
    printf("br_ip:%s\n", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));
    bzero(buff, sizeof(buff));
    sprintf(buff, "%s", (char*)inet_ntoa(((struct sockaddr_in*)
(&buf[if_len].ifr_addr))->sin_addr));

```

```

        inet_pton(AF_INET, buff, &ip);
        memcpy(net_interface[if_len].br_ip, &ip, 4);
    }else{
        char str[256];
        sprintf(str, "SIOCGIFADDR ioctl %s", buf[if_len].ifr_name);
        perror(str);
    }

    /*MAC地址 */
    if (!(ioctl(sock_raw_fd, SIOCGIFHWADDR, (char *) &buf[if_len]))){
        printf("MAC:%02x:%02x:%02x:%02x:%02x:%02x\n\n",
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[0],
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[1],
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[2],
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[3],
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[4],
            (unsigned char) buf[if_len].ifr_hwaddr.sa_data[5]);
        memcpy(net_interface[if_len].mac, (unsigned char
*)buf[if_len].ifr_hwaddr.sa_data, 6);
    }else{
        char str[256];
        sprintf(str, "SIOCGIFHWADDR ioctl %s", buf[if_len].ifr_name);
        perror(str);
    }
}
} //-while end
close(sock_raw_fd);    //关闭socket
}

```

编写makefile文件

```

OBJ += main.o
OBJ += arp_pthread.o
OBJ += arp_link.o
OBJ += ip_pthread.o
OBJ += ip_link.o
OBJ += key_pthread.o
OBJ += get_interface.o

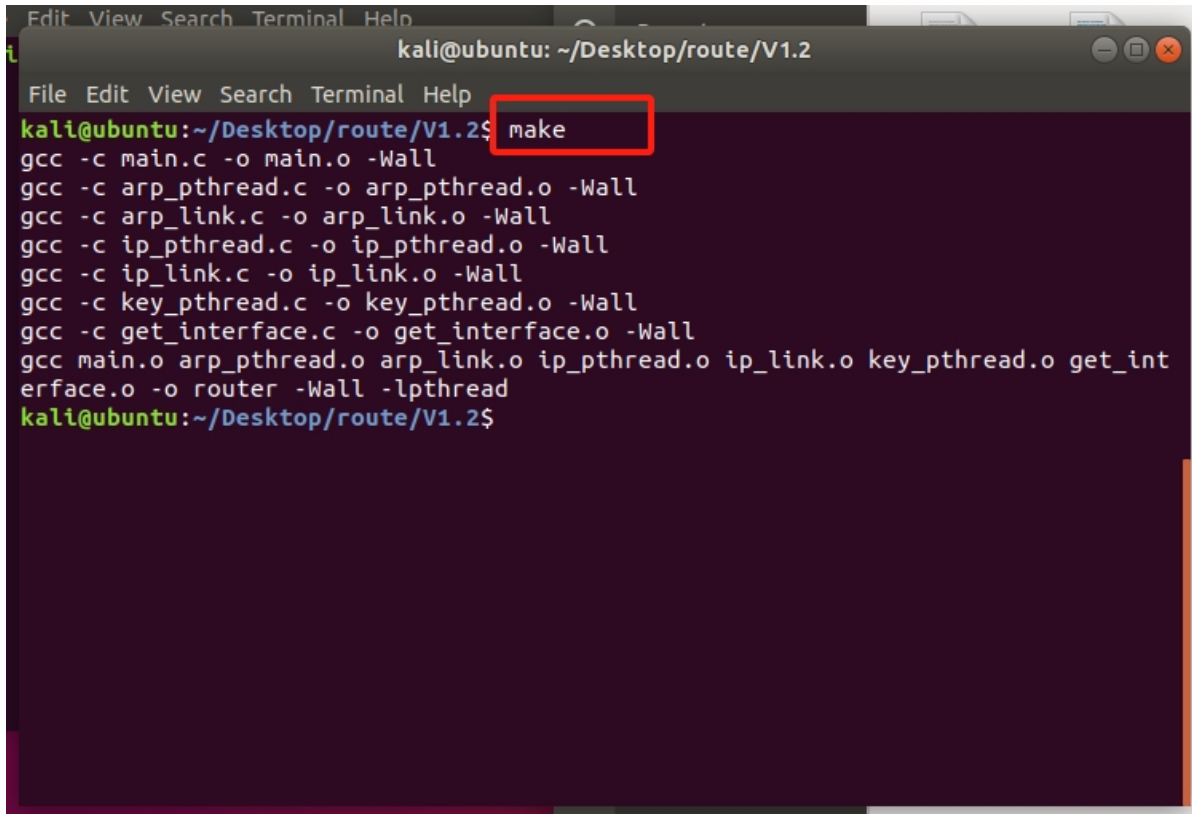
FLAGS = -Wall
CC = gcc

router:$(OBJ)
    $(CC) $(OBJ) -o $@ $(FLAGS) -lpthread
%.o:%.c
    $(CC) -c $^ -o $@ $(FLAGS)
.PHONY:clean
clean:
    rm router *.o -rfv

```

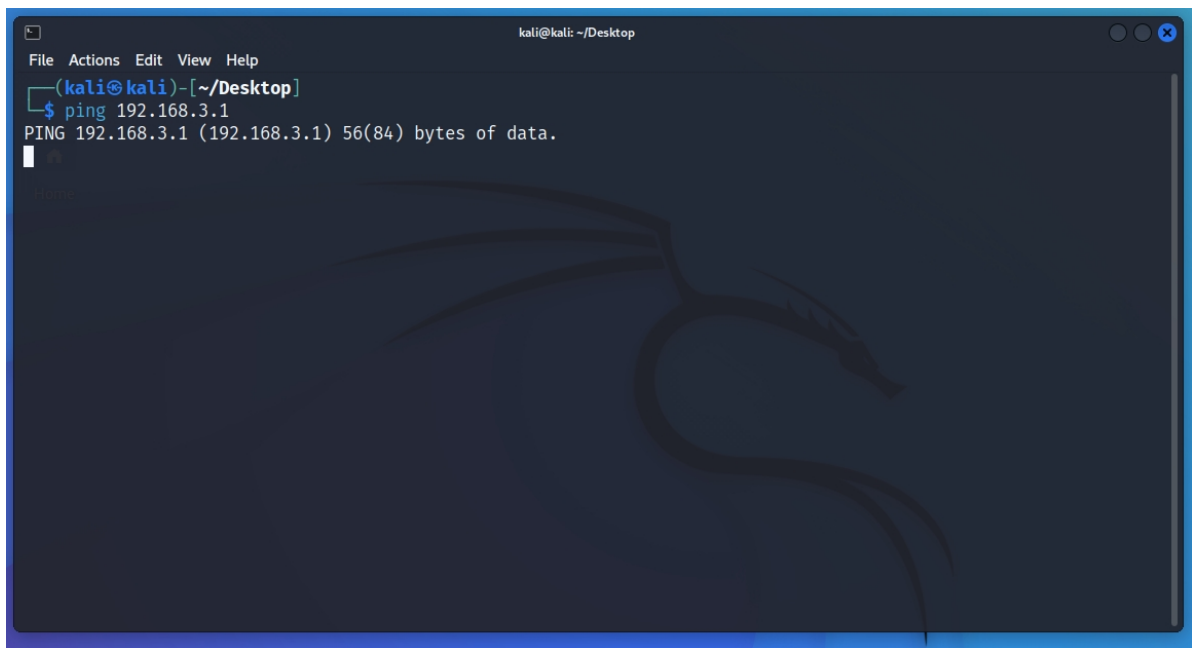
4.实验效果

先编译路由项目



```
kali@ubuntu: ~/Desktop/route/V1.2
File Edit View Search Terminal Help
kali@ubuntu:~/Desktop/route/V1.2$ make
gcc -c main.c -o main.o -Wall
gcc -c arp_thread.c -o arp_thread.o -Wall
gcc -c arp_link.c -o arp_link.o -Wall
gcc -c ip_thread.c -o ip_thread.o -Wall
gcc -c ip_link.c -o ip_link.o -Wall
gcc -c key_thread.c -o key_thread.o -Wall
gcc -c get_interface.c -o get_interface.o -Wall
gcc main.o arp_thread.o arp_link.o ip_thread.o ip_link.o key_thread.o get_int
erface.o -o router -Wall -lpthread
kali@ubuntu:~/Desktop/route/V1.2$
```

编译成功，先不启动，尝试使用客户端互相ping



```
kali@kali: ~/Desktop
File Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ping 192.168.3.1
PING 192.168.3.1 (192.168.3.1) 56(84) bytes of data.

```

发现ping不通

启动路由器，需要注意，启动路由器，需要使用root用户

```
root@ubuntu: /home/kali/Desktop/route/V1.2
File Edit View Search Terminal Help
kali@ubuntu:~/Desktop/route/V1.2$ make
gcc -c main.c -o main.o -Wall
gcc -c arp_pthread.c -o arp_pthread.o -Wall
gcc -c arp_link.c -o arp_link.o -Wall
gcc -c ip_pthread.c -o ip_pthread.o -Wall
gcc -c ip_link.c -o ip_link.o -Wall
gcc -c key_pthread.c -o key_pthread.o -Wall
gcc -c get_interface.c -o get_interface.o -Wall
gcc main.o arp_pthread.o arp_link.o ip_pthread.o ip_link.o key_pthread.o get_int
erface.o -o router -Wall -lpthread
kali@ubuntu:~/Desktop/route/V1.2$ su
Password:
root@ubuntu: /home/kali/Desktop/route/V1.2#
```

./router 运行路由器

```
root@ubuntu: /home/kali/Desktop/route/V1.2
File Edit View Search Terminal Help
gcc -c get_interface.c -o get_interface.o -Wall
gcc main.o arp_pthread.o arp_link.o ip_pthread.o ip_link.o key_pthread.o get_int
erface.o -o router -Wall -lpthread
kali@ubuntu:~/Desktop/route/V1.2$ su
Password:
root@ubuntu: /home/kali/Desktop/route/V1.2# ./router

-----ip_link_start-----
172.20.226.6
172.20.226.5
-----ip_link_end-----

interface_num=3

ens37
-2-ens37--
UP
IP:192.168.2.254
netmask:255.255.255.0
br_ip:192.168.2.255
MAC:00:0c:29:ad:24:47

ens33
-1-ens33--
```

尝试客户端ping

```
kali@kali: ~/Desktop
File Actions Edit View Help
(kali@kali)~$ ping 192.168.3.1
PING 192.168.3.1 (192.168.3.1) 56(84) bytes of data:
64 bytes from 192.168.3.1: icmp_seq=1 ttl=64 time=1.33 ms
64 bytes from 192.168.3.1: icmp_seq=2 ttl=64 time=1.77 ms
64 bytes from 192.168.3.1: icmp_seq=3 ttl=64 time=1.53 ms
64 bytes from 192.168.3.1: icmp_seq=4 ttl=64 time=1.02 ms
```

数据通过了